

Backend plan: registration, Psoola payments, tickets by email, and the database

A plain-language plan for turning the iSmile website into a working registration system: people register, pay through Psoola (Visa, MasterCard, FIB, FastPay), receive their ticket by email, and every registration lands in a database that you can see inside your own admin.

Recommendation in one line: build our own small backend (PHP + MySQL) on the server you are buying, connected to Psoola's single API and webhook. Do **not** use Pretix for the 2026 summit — it can technically take Psoola only through a custom plugin on a self-hosted Pretix server, and it would put the attendee database outside your admin. Section 3 explains.

Prepared 27 September 2026 · Replaces the plan of 23 September 2026

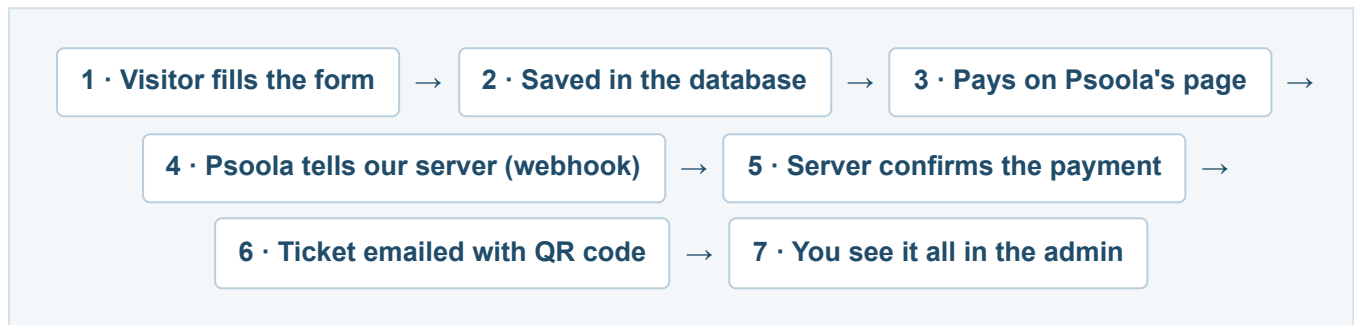
Planning document — no code has been written for the items in this plan. Items marked *Confirm* need an answer from Psoola or from you before building.

1. What we are building

Today the website has the whole visitor journey on screen — registration form, lunch options, payment choice, success page, sponsor form — but **nothing behind it**. The registration page shows a reference number that is made up in the browser, no money is taken, no email is sent, and no one's details are saved anywhere.

Sponsor requests open WhatsApp or email.

The finished system does this, automatically, in all three languages:



Three rules run through every part of this plan:

- **No successful payment, no ticket.** A ticket is created only after our server has confirmed with Psoola that the money arrived. Not when the visitor says so, not when the browser comes back from the payment page.
- **The server decides prices and status.** The browser only shows them. Nobody can register for 0 IQD by editing the page.
- **Everything is in your admin.** Registrations, student ID photos, payments, tickets, sponsor and booth requests — one login, one place, exportable to Excel.

What exists today and what changes

Part	Today	After this plan
Registration form (3 steps, EN/AR/KU)	Works on screen; sends nothing	Same screens; the last step sends everything to our server and goes to Psoola
Student ID photo	Chosen in the form, then thrown away	Uploaded to a private folder; shown to you in the admin for approval
Payment	"We send your payment link by email" (by hand)	Automatic, on Psoola's hosted page; card, FIB or FastPay
Ticket	None	Emailed automatically after payment: ticket number, QR code, PDF attached
Reference number	Made in the browser (ISM26-XXXXXX), cannot be looked up	Made by the server, same format, stored, searchable in the admin
Admin	Edits site text and lists; saves through a GitHub key	Same admin, plus: Registrations, Payments, Sponsors & booths, Dashboard; saves directly on the server

Sponsor / booth requests	Opened in WhatsApp or email	Saved in the database, tracked in the admin (section 7)
Hosting	GitHub Pages (free, no server)	Your own domain and server (section 8)

2. Payments with Psoola

Who they are. Psoola is the payment company the organiser already works with. They combine several payment methods behind one merchant API and will give us that API together with a webhook. For iSmile it will take exactly four payment methods: **Visa, MasterCard, FIB and FastPay** — nothing else. For us this is the best possible shape: **one contract, one API, one webhook**, and the four methods our visitors actually use.

Two words you will hear a lot:

- **API** — the address our server calls to say "create a payment of 150,000 IQD for reference ISM26-K7P2QX" and to ask "is payment 12345 paid?".
- **Webhook** — the opposite direction: an address on *our* server that Psoola calls to say "payment 12345 succeeded" (or failed). This is how the ticket goes out without anyone watching a screen.

How one payment will work

#	Step	Detail
1	Visitor finishes the form and presses Pay	Our server saves the registration as unpaid , works out the total itself (ticket + lunch days, from the prices in the admin), and asks Psoola to create a payment for that amount.
2	Visitor is sent to Psoola's payment page	They choose Visa or MasterCard, FIB, or FastPay <i>there</i> . Our site never sees card numbers or bank logins — Psoola handles that, which keeps us out of card-security rules.
3	Visitor comes back to our site	They see "Checking your payment...". This page never decides anything; it only waits for step 4.
4	Psoola calls our webhook	Our server checks the message really came from Psoola (signature or secret), then asks Psoola's API directly for the payment status rather than trusting the message alone.
5	Paid	Registration becomes paid , a ticket number and QR code are created, the ticket email is queued. The waiting page turns into "You're in — check your email".
6	Failed, cancelled or abandoned	Registration stays unpaid . No ticket. The visitor gets a "Pay now" link by email that starts a fresh payment for the same registration — they do not fill the form again.

Things that go wrong in real life, and what we do

- **The webhook arrives twice** (this is normal for every payment provider). The second one is ignored: a payment can only be confirmed once, and a ticket is never made twice for the same registration.

- **The webhook arrives late or never.** Every 10 minutes a small scheduled job asks Psoola about payments that are still "waiting" and closes them either way. A visitor whose bank was slow still gets the ticket.
- **The visitor closes the page mid-payment.** Nothing is lost; their registration is unpaid and the "Pay now" email covers it.
- **Someone tries to pay 1,000 IQD for a 150,000 IQD ticket.** Impossible: the amount is set by our server, and the confirmed amount from Psoola is compared with the expected one before the ticket is issued.
- **No seat-hold timers** (as decided earlier): an unpaid registration simply stays unpaid.

What we need from Psoola — ask for all of it now

This list is the **critical path** of the whole project. Nothing in section 2 can be tested until it arrives, and merchant onboarding with any payment company usually takes weeks, not days.

Item	Why
Merchant account in the name of the organiser (italk / iSmile)	The money has to land somewhere. Ask what company documents they need.
API documentation (the PDF or web page developers read)	Tells us how to create a payment, how the redirect works, and what the webhook sends.
Test ("sandbox") credentials and test cards / test wallets	So we can make hundreds of fake payments while building, without real money.
Live credentials — later, only when we go live	Kept on the server only, never in the website files.
How they secure the webhook (signature, secret, or IP list)	Otherwise anyone who guesses the address could pretend a payment succeeded.
Confirmation that all four methods — Visa, MasterCard, FIB, FastPay — are live on our account from day one	The payment step on our site will list exactly these four. If one is delayed, we hide it until it works.
Currency (IQD), minimum and maximum amounts, and their fees	Fees decide the ticket price; maximums matter for sponsors if they ever pay online.
Refund: possible through the API, by request, or not at all?	Tickets are non-refundable, but a duplicate payment must be refundable.
Settlement: when and how the money reaches the bank account; a statement or report we can download	Needed to reconcile with the admin's payment list.
Whether the payment page supports Arabic and Kurdish	Most visitors register in Arabic or Kurdish. If the page is English-only, we tell them beforehand.

Confirm: Psoola has said they will generate the API for us and hand it over. Get that in writing with a date, together with the documentation and the test credentials — the build cannot start on the payment side until they arrive, and every week they slip moves the registration opening by a week.

3. Can Pretix help? — the answer

Pretix can be made to work with Psoola, but only on our own Pretix server with a plugin we write ourselves — and it does not give you the database inside your admin. For the 2026 summit it adds more work than it removes. Recommendation: not now.

The question was: can Pretix handle "one API plus a webhook, no ticket without a successful payment"? Here is the honest picture.

Point	What we found
Does Pretix already support Psoola, FIB or FastPay?	No Pretix ships Stripe, PayPal, SEPA, bank transfer and others — no Iraqi provider.
Can a new provider be added?	Yes, as a plugin Pretix documents a "payment provider plugin" (a Python class that redirects to the gateway and confirms the payment). Psoola's redirect-and-webhook style fits that model.
Can we use that plugin on pretix.eu (their hosted service)?	Not on the standard hosted plan pretix.eu's hosted service does not let customers upload their own plugins. A custom plugin means running Pretix on our own server (Python, PostgreSQL, Redis, a worker — a second, heavier system next to the website) or paying for their Enterprise arrangement. <i>Confirm with sales@pretix.eu before ever choosing this route.</i>
Kurdish language and right-to-left layout	Kurdish is not available We would translate Pretix ourselves. Arabic exists; full right-to-left layout is uncertain. Our own site already does all three properly.
"The database inside my admin"	No With Pretix, attendees live in Pretix's own admin. We would build screens that copy them into ours — extra work to end up where the custom backend starts.
Sponsors and exhibition booths	Not covered Pretix sells tickets; sponsor and booth requests need our own screens either way.
Time until 20 November	Eight weeks Setting up a self-hosted Pretix, writing and testing a payment plugin, translating to Kurdish, then still building the admin screens does not fit.
What Pretix would give us for free	Real things A check-in phone app with QR scanning, a waiting list, invoices, and a battle-tested order system. Worth revisiting for iSmile 2027 if you want the check-in app — the database we build now can be exported into it.

Decision: our own backend, described in the rest of this plan. It is smaller than Pretix, it speaks Psoola directly, it keeps the three languages we already have, and it lives inside the admin you already use.

4. Tickets by email

When a payment is confirmed, the visitor receives one email within a minute. It contains:

- Their name exactly as typed (it will also be the name on the certificate), the ticket type (Professional / Student), lunch days if bought, and the reference (ISM26-XXXXXX).
- A **QR code** shown in the email and a **PDF ticket** attached, both carrying a unique ticket number. Scanning the QR at the door looks the ticket up in the database, so a screenshot cannot be used twice.
- The venue, dates, and a link to the workshops page. In the visitor's chosen language.

Rules for sending

Rule	Meaning
Never twice	Each ticket has a send status: pending → sent or failed . A sent ticket is not sent again by accident.
Retry	A failed send is retried 3 times over an hour, then shows as failed in the admin.
Resend button	In the admin, next to each registration — for "I deleted the email" and "I typed my address wrong" (you fix the address, then resend).
Test mode	Until launch, every email goes to your own address, whatever address is on the form.
Failed email ≠ failed ticket	If the email cannot be delivered, the ticket still exists and is visible in the admin. The person can be found by name or phone at the door.

Which email service

Not a personal Gmail account. Gmail caps at roughly 500 emails a day, and a personal account sending tickets to two thousand strangers gets flagged as spam. Use a transactional email service — **Brevo** (free up to 300 emails/day, cheap above that) or **Amazon SES** — sending from `tickets@ismile.krd` on your own domain, with the domain records (SPF, DKIM) set so the emails land in the inbox, not in spam. The sending code is written as one swappable piece, so changing service later is an afternoon, not a rebuild.

Certificates of attendance (with the same name) are a separate, later feature: generated after the summit from the check-in list. Nothing in this plan blocks it.

5. The database

MySQL on your server. One row per registration, one per payment attempt, one per ticket, one per sponsor request — linked together so that from any registration you can see its payments, ticket and student check, and from any payment you can find the person. Nothing is deleted; things are marked instead, so there is always a history.

Table	What it holds	Notes
registrations	Reference (ISM26-...), first / father's / grandfather's name, phone (normalised to +964...), email, city, gender, age, specialty, ticket type: professional or student , lunch day 1 / day 2, language used, status (unpaid · paid · cancelled), created and paid times	Every field on today's form, nothing more. Age and gender: confirm you actually need them; if not, drop them and collect less.
student_checks	University and department, ambassador code, ID photo file (private path), review status (waiting · approved · rejected), reviewer, reason, review time	Only for student registrations. Photos are stored <i>outside</i> the public web folder and served only to a logged-in admin.
payments	Registration, Psoola payment ID, method used (card / FIB / FastPay), amount expected, amount confirmed, currency, status (created · waiting · paid · failed · refunded), the raw Psoola messages	One registration can have several attempts (a failed one, then a paid one). Exactly one can be "paid".
tickets	Registration, ticket number, QR payload, email status (pending · sent · failed), send attempts, checked-in time	Created only from a paid payment. Used by the door scan.
sponsor_requests	Kind (sponsorship / exhibition booth), package or tier, company, contact name, phone, email, message, status, assigned person, notes, created time	Section 7.
ambassadors	Code, owner name, university, count of registrations that used it	Today a code is free text; here it becomes a real list so you can count per ambassador.
webhook_log	Every message Psoola sent us, when, and what we did with it	The first place to look when someone says "I paid but got nothing".
admin_users and audit_log	Who can log in and with which role; who approved, rejected, resent, exported or refunded what, and when	The existing admin accounts move here from the JSON file.

Privacy and keeping

- Student ID photos are sensitive. They are visible only to admins with the "registration" role, and are deleted a fixed time after the summit (suggest 3 months) — the approval decision stays, the photo goes.
- Daily automatic backup of the database and the photo folder to somewhere *other* than the server (the hosting's backup, or a cloud bucket). Test one restore before launch.
- Exports (Excel / CSV) are logged with who exported and when.

6. The admin: what you will see

Your current admin stays as it is for site content. It gains new pages, in the same style, behind the same login (moved to the server, so it is a real login rather than a browser check):

Page	What you can do
Dashboard	Big numbers: registered, paid, professionals vs students, students waiting for approval, lunch counts per day, money confirmed, tickets sent / failed, open sponsor requests.
Registrations	The list. Search by name, phone, email or reference. Filter: professional / student, paid / unpaid, approved / waiting, lunch, city, language. Open one to see every detail, the payment history and the ticket. Buttons: <i>Resend ticket</i> , <i>Send "Pay now" link</i> , <i>Edit email or phone</i> (logged), <i>Cancel</i> . <i>Export to Excel</i> with the current filter.
Student approvals	Students waiting, oldest first. See the ID photo large, the university typed, the name. <i>Approve</i> or <i>Reject with a reason</i> (the reason goes to the student by email).
Payments	Every attempt with its Psoola ID, method, amount and status. A daily total to compare with Psoola's settlement report. Refund marking.
Sponsors & booths	Section 7.
Check-in (summit days)	A simple phone-friendly page: scan the QR or type a name, mark as arrived, see "already checked in" if scanned twice. Works on the venue Wi-Fi from any phone; a printed list as the fallback.
Settings	Ticket and lunch prices (already exist), registration open / closed, test mode on / off for emails, Psoola live / test switch (the secrets themselves stay in a file on the server, not on this page).

Roles

Owner (everything), Registration (registrations, approvals, resend, export), Finance (payments, refunds, exports), Content (the existing site editing), Check-in (only the scan page). Every action is checked on the server, not just hidden on screen.

7. Sponsors and exhibition booths — handle with care

You asked for care here, and it is deserved: a sponsor is worth many tickets, the amounts are large, and a mistake is public. The rule is simple: **sponsorship and booths are never sold like tickets**. They are requests that a person handles, with the database keeping the record.

Stage	What happens
New	The sponsor form (already on the site) saves the request to the database instead of opening WhatsApp. The company gets an automatic "we received your request" email; you get a notification email.
Contacted	Someone on the team calls or emails. Notes are kept on the request. Assigned to a person so two people do not both call.
Agreed	Package, booth position and price agreed. An invoice is sent from the admin (a PDF with the iSmile details, a number and a due date).
Paid	Marked by Finance when the money arrives (bank transfer, or an Psoola payment link if Psoola's limits allow the amount — <i>confirm</i>).
Confirmed	Only now does the sponsor's logo appear on the website, from the existing sponsors list in the admin, and the booth spot is counted as taken.
Declined / withdrawn	Kept with a reason, so next year you know the history.

The mistakes this design prevents

- **Selling the same booth twice.** Each tier has a number of spots (already in the admin). The admin shows spots taken vs available and refuses to confirm one more than exists.
- **Publishing a logo before the money arrives.** A request cannot move to Confirmed without being Paid first (the Owner can override, and it is logged).
- **A company paying a ticket price by mistake.** The sponsor form never leads to the ticket payment page. If a sponsor wants to attend, their staff register as normal attendees with normal tickets — or you issue complimentary tickets from the admin, marked as such.
- **Losing a request in a WhatsApp thread.** Every request is in the list with a status and a date; the dashboard shows how many are open and how old the oldest is.

8. The server and the domain you are buying

You will buy these yourself. Here is exactly what to buy, and what to check before paying. The website today is plain files; the backend in this plan needs a place that can run code, hold a database, keep files private, and be reachable by Psoola's webhook at a fixed address.

Domain

- The site already uses `info@ismile.krd`, so **ismile.krd** is the natural name; if it is not available, **ismile.iq** or **ismile-summit.com**. Buy it in the organiser's name with your own email as contact, for at least two years,

and turn on auto-renew — an expired domain in November would take the whole summit offline.

- You also get email addresses on it: `info@`, `tickets@`, `sponsors@`. The ticket emails will be sent "from" `tickets@ismile.krd`.

Server: two sensible choices

A · Managed web hosting with cPanel

Recommended

Shared or "cloud" hosting with a control panel. PHP, MySQL, email, SSL certificate, daily backups and cron jobs are all ready; nobody has to maintain Linux.

Roughly \$5–20 per month. Enough for this: a few thousand registrations and a few hundred simultaneous visitors is a small load.

Choose a plan that states: PHP 8.1 or newer, MySQL 8 / MariaDB 10.6+, free SSL (Let's Encrypt), cron jobs, at least 10 GB storage, daily backups kept for 30 days, SSH access.

B · A VPS (your own virtual server)

2 vCPU, 4 GB RAM, 40 GB SSD, Ubuntu 24.04, with a panel (cPanel, Plesk or CloudPanel) installed on it. More control and room to grow.

Roughly \$10–30 per month plus the panel licence if any.

Downside: someone must update it and watch it. Only pick this if that person exists. A VPS without a panel is not recommended for this team.

Either way: a provider with servers in Europe or the Gulf is fine; latency from Iraq is a few tens of milliseconds. Avoid free tiers and one-year promotional prices that jump later.

What we do with it in the first week

1. Point the domain at the server; turn on HTTPS (free certificate). The address becomes `https://ismile.krd`.
2. Upload the website. The existing `server/admin.php` replaces the GitHub-key saving, so the admin saves instantly on the server (this was already planned and is written; it has never run on real hosting, so expect one round of small fixes).
3. Create the MySQL database and the private folder for ID photos (outside the public web folder).
4. Set up a **staging copy** at `test.ismile.krd` with Psoola's test credentials, so every feature is tried there before it touches the real site.
5. Give Psoola the webhook address (`https://ismile.krd/api/psoola-webhook`) — they need it to send confirmations.

9. Technology choice

PHP 8 + MySQL, on the hosting above, keeping the current website exactly as it is. Reasons, in order:

- It is what managed hosting runs natively — no extra services, no server administrator needed.
- The admin's server half (`server/admin.php`) is already PHP; the new parts sit next to it and share its login.
- The front end (forms, three languages, right-to-left, light/dark themes) stays. Each form gains one "send to server" call; nothing is rebuilt.
- Any local developer can maintain PHP + MySQL later.

The 23 September plan proposed Django + PostgreSQL + Redis + Pretix. That was right for a larger team and a longer runway; with Psoola giving one API, a managed server, and eight weeks, it is more machinery than the job needs. If the summit grows to tens of thousands of attendees, the database designed in section 5 moves to a bigger stack without changing what it stores.

10. Security essentials

- HTTPS everywhere; the admin refuses plain HTTP.
- Psoola secrets, database password and email keys live in one file on the server outside the web folder — never in the website files, never in GitHub.
- Every form field is checked again on the server (length, format, allowed values, file type by content not by name, 8 MB limit).
- Rate limits on the registration form, the "Pay now" link and the admin login; admin login locks for 15 minutes after 5 wrong tries; the Owner account gets two-factor codes.
- Webhook: signature verified, then the status re-fetched from Psoola's API; unknown or unsigned calls are logged and ignored.
- ID photos and exports are never at a public address; they are served through the admin's login.
- Logs never contain passwords, card details or full ID photos.

11. Build order and timing

The summit is **20–21 November 2026**. Working back: registration should be open with payments by **mid-October** to give people five weeks. That is realistic *only* if Psoola's test credentials arrive in the first two weeks — everything in the payment column below waits on them. Start that conversation today.

Week	Work	Done when
1 29 Sep – 5 Oct	Buy domain and hosting. Move the site. Admin saves through the server. Database created. Ask Psoola for the merchant account, documentation and test credentials.	Site is live at ismile.krd with HTTPS; you can edit and save from the admin; the Psoola request is sent in writing.

2 6 – 12 Oct	Registration saves to the database, with the reference made by the server. Student ID upload to the private folder. Admin: Registrations list and Student approvals page. Sponsor form saves to the database; Sponsors & booths page.	A test registration on the staging site appears in the admin with its photo; you can approve it; a sponsor request appears in its list.
3 13 – 19 Oct	Psoola payment: create, redirect, return page, webhook, status check job, "Pay now" link. Payments page in the admin.	On staging: a test card payment turns a registration paid; a failed one does not; a webhook sent twice changes nothing; a "missed" webhook is caught by the check job.
4 20 – 26 Oct	Ticket email with QR and PDF, in three languages. Send status, retries, Resend button, test mode. Email domain records so mail reaches the inbox.	A paid test registration receives its ticket within a minute; a resend works; a wrong address shows as failed in the admin.
5 27 Oct – 2 Nov	Dashboard, exports, roles, audit log, check-in page. Go-live checklist: real Psoola credentials, one real payment of a small amount refunded, backups tested.	Registration opens to the public. (Later than mid-October — see the note below.)
6–7 3 – 16 Nov	Watch the first real registrations daily. Fix what real people find. Prepare the check-in devices and the printed fallback list. Train two people on approvals and check-in.	No failed tickets outstanding; every student reviewed within 24 hours; check-in rehearsed with 20 test tickets.
Summit 20 – 21 Nov	Check-in at the door; registrations page for questions at the desk; one person on call for the system.	Certificates generated from the check-in list afterwards.

About the dates: this order puts payments before emails on purpose — if Psoola is late, weeks 2 and 4 still happen and registration can open with "pay by link" as a temporary fallback, exactly as the site says today, and switch to automatic payment the day Psoola is ready. Opening registration around **25 October** is the honest target; mid-October needs Psoola credentials by 6 October.

Tests before anyone real pays

- Two people paying at the same second for the same registration — only one ticket.
- Webhook replayed, delayed 20 minutes, and never sent.
- A student rejected, then re-uploading a better photo.
- Wrong email typed, fixed in the admin, ticket resent.
- Arabic and Kurdish end to end, including the ticket email and PDF.
- A phone on a slow connection uploading an 8 MB photo.
- Restore last night's backup onto the staging site.
- Someone trying to open another person's ticket or photo by guessing an address.

12. Decisions I need from you

#	Decision	Suggestion
1	Psoola: send the request for merchant account, documentation and test credentials.	This week. Ask for the written answer to the "open to private organisers?" question at the same time.
2	Domain name.	ismile.krd if available.
3	Hosting: A (managed cPanel hosting) or B (VPS).	A , unless someone on the team will maintain a server.
4	Ticket prices and lunch prices in IQD.	Needed before registration opens; the admin already has the fields.
5	Students: approve the ID <i>before</i> they pay, or let them pay first and refund if rejected?	Approve first. A student registers, you approve (usually same day), they get the "Pay now" email, they pay, ticket goes out. Professionals pay immediately. Refunds are the messiest part of any payment system; this avoids most of them.
6	Refund policy in words, for the form and the ticket email.	"Tickets are non-refundable" (as today), plus "duplicate payments are refunded".
7	Do you need age and gender?	If they are not used for anything, remove them. Less data, less risk.
8	Who reviews student IDs, and who handles sponsor requests?	One named person each, with a backup. They become the first admin accounts with those roles.
9	Email service: Brevo or Amazon SES.	Brevo — simpler to set up, free tier covers testing.
10	Booth spots per tier, and whether sponsors may ever pay through Psoola.	Spots are already in the admin; confirm the numbers. Sponsor payment by bank transfer unless Psoola's limits clearly allow it.

13. Running costs

Item	Cost	Note
Domain	\$10–40 per year	.krd and .iq vary; .com about \$12.
Hosting (option A)	\$5–20 per month	Includes SSL, email boxes, backups.
Email sending (Brevo)	\$0–25 per month	Free up to 300/day; 2,000 tickets in a launch week may need one paid month.
Psoola fees	<i>Confirm</i>	Usually a percentage per payment. Build it into the ticket price.
Backups off-server	\$0–5 per month	Often included with hosting; a cloud bucket otherwise.
Pretix	\$0	Not used this year.

References

- Pretix payment-provider plugins: docs.pretix.eu — [Writing a payment provider plugin](#) · Pretix hosted pricing: pretix.eu/about/en/pricing · self-hosted: pretix.eu — [self-hosted](#)
- Earlier documents in this repository: [docs/ismile-backend-plan.pdf](#) (23 Sep 2026, Django + Pretix proposal) and [docs/pretix-requirements.md](#) (24 Sep 2026, requirements R1–R12 — still the definitive list of what the forms collect).